Lab - Continuous Integration with Jenkins

In this lab you will work with Jenkins which is a Continuous Integration (**CI**) tool.

The project supplied is written in Java, but Jenkins' CI pipeline can work for any buildable software.

Objectives

- To get Jenkins up and running using Docker
- To explore the options Jenkins gives us for connecting to other services
- To explore the Jenkins dashboard

Prerequisites

- Docker desktop is installed 
- Git CLI
- You have a GitHub account

Objective of this exercise



Write code
Make changes
Check the code in

Detects the change
Raises a build event

Build and runs tests
Deploy the code
Report result to GitHub

Production

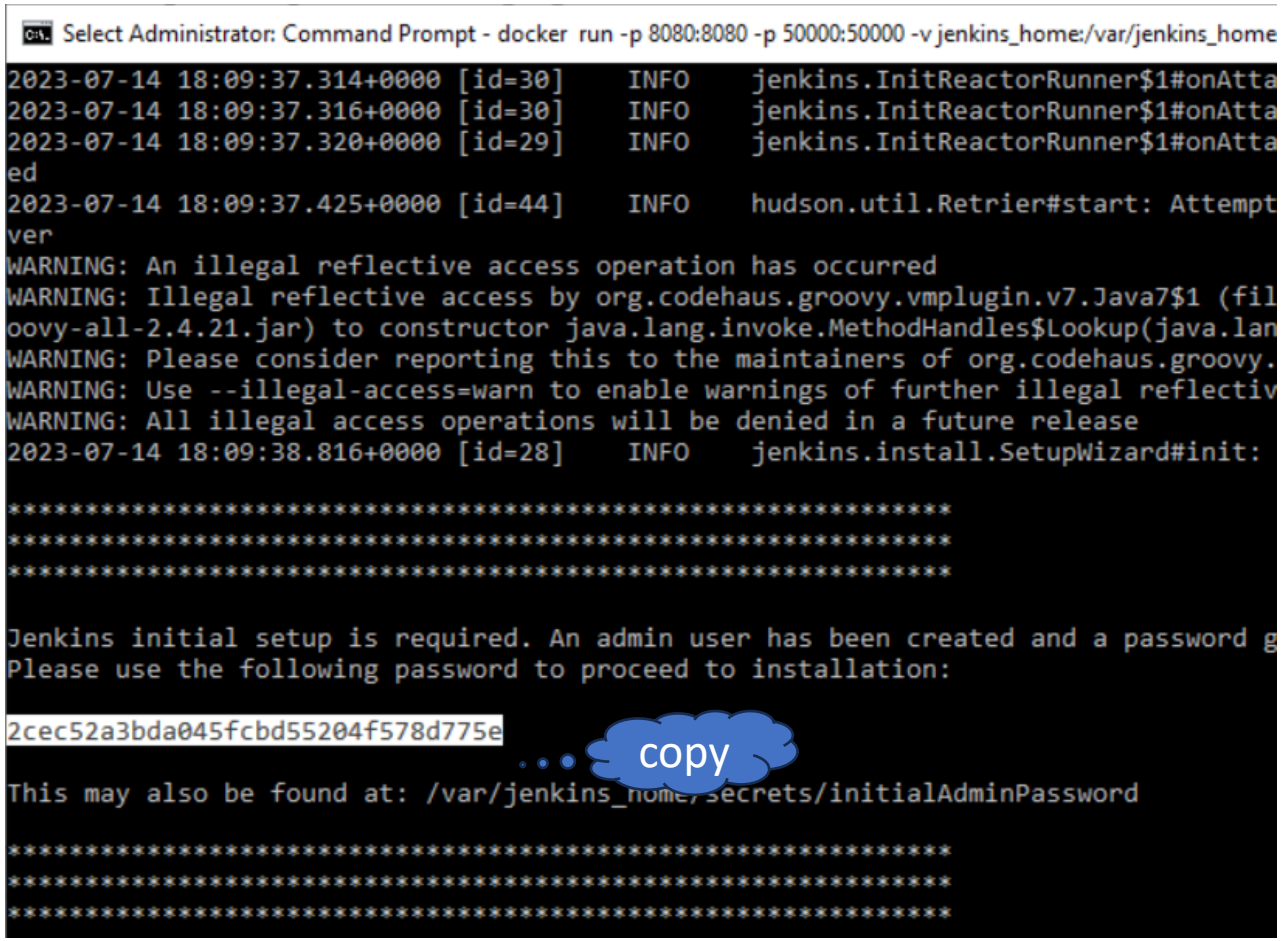
In this lab, you will

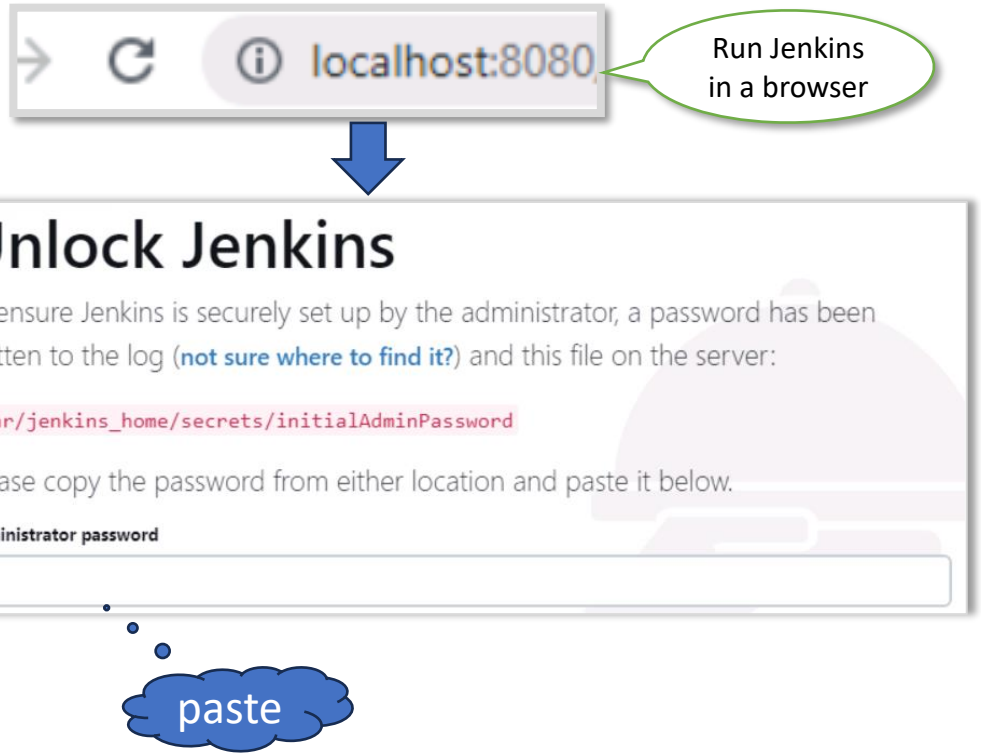
- Deploy a Maven app from GitHub to Jenkins
- Build and test the app using Jenkins
- Make changes to your code and deploy the changes to GitHub
- Set up Jenkins to detect the change in code and rebuild the app

Run Jenkins using Docker

In a terminal window, type

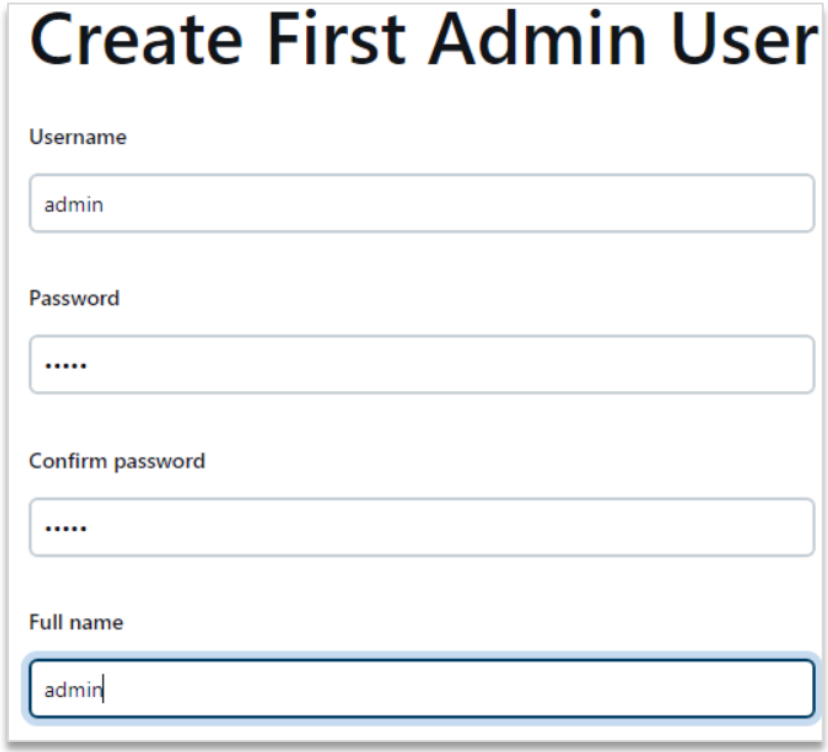
```
docker run -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```





Jenkins

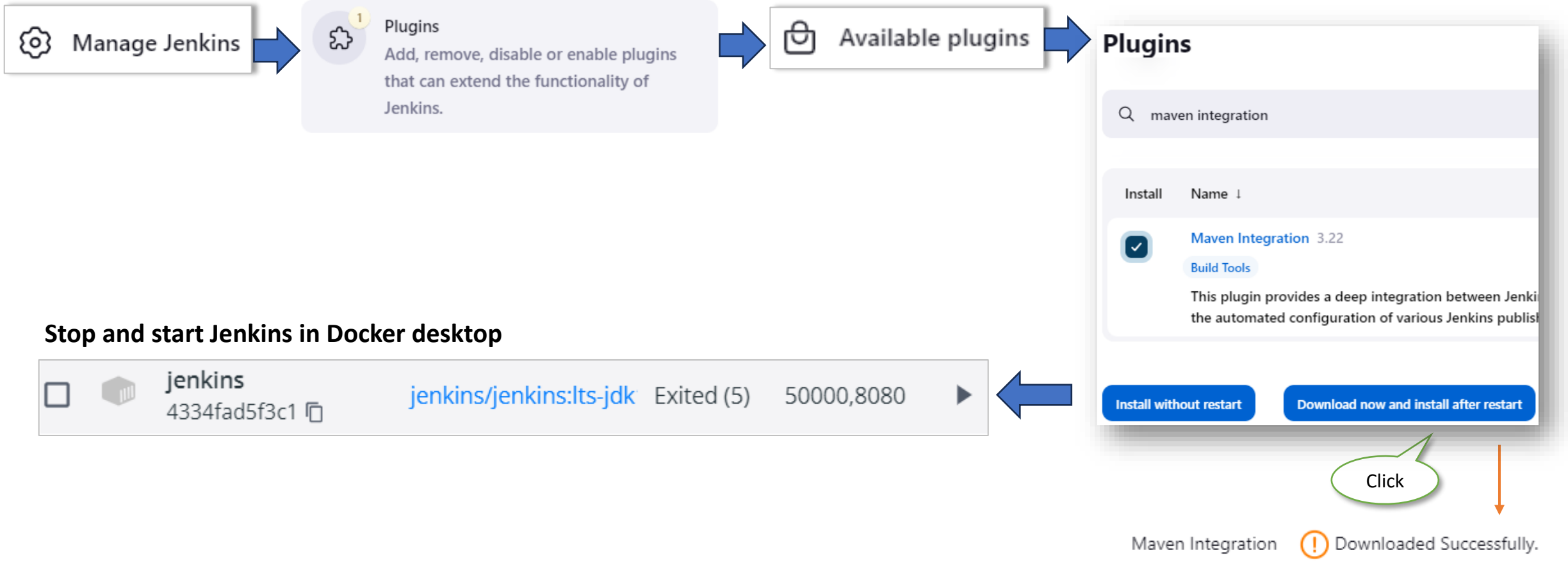






Accept or change the port

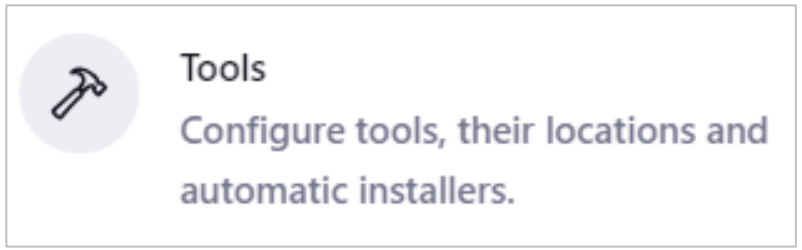
Jenkins



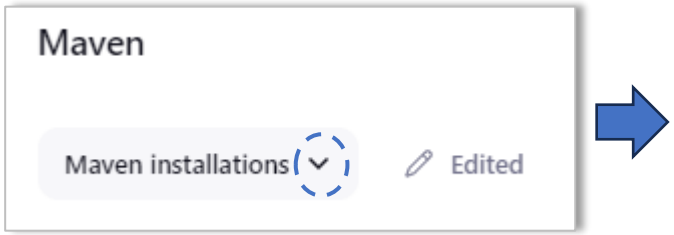
Stop and start Jenkins in Docker desktop

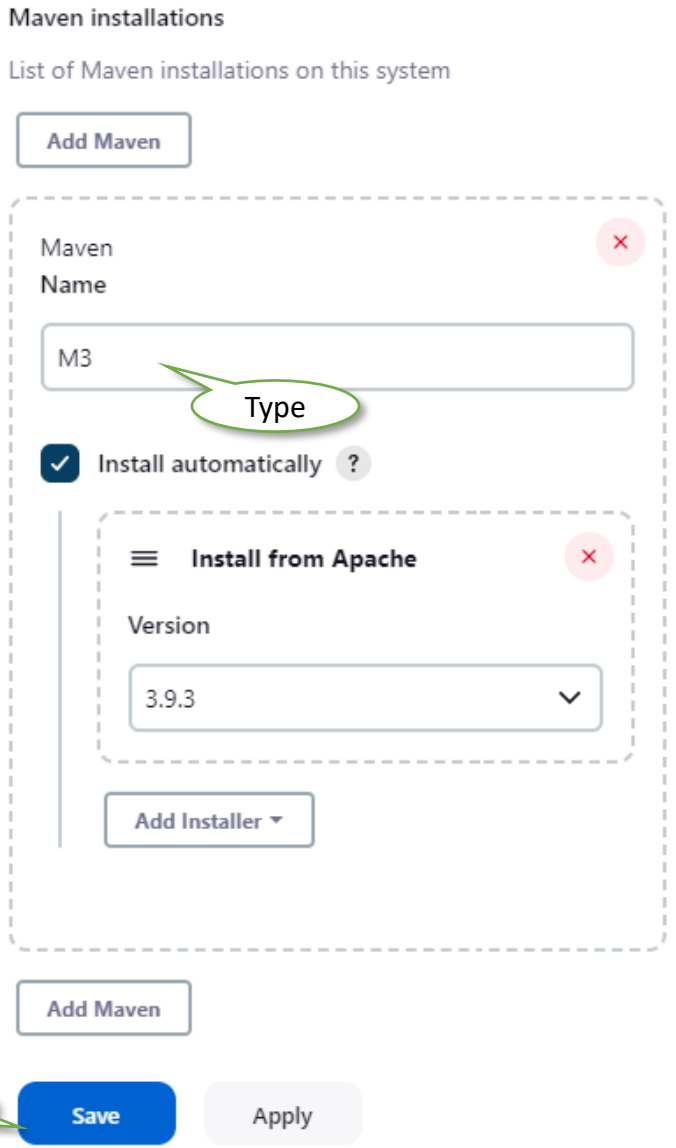
Jenkins



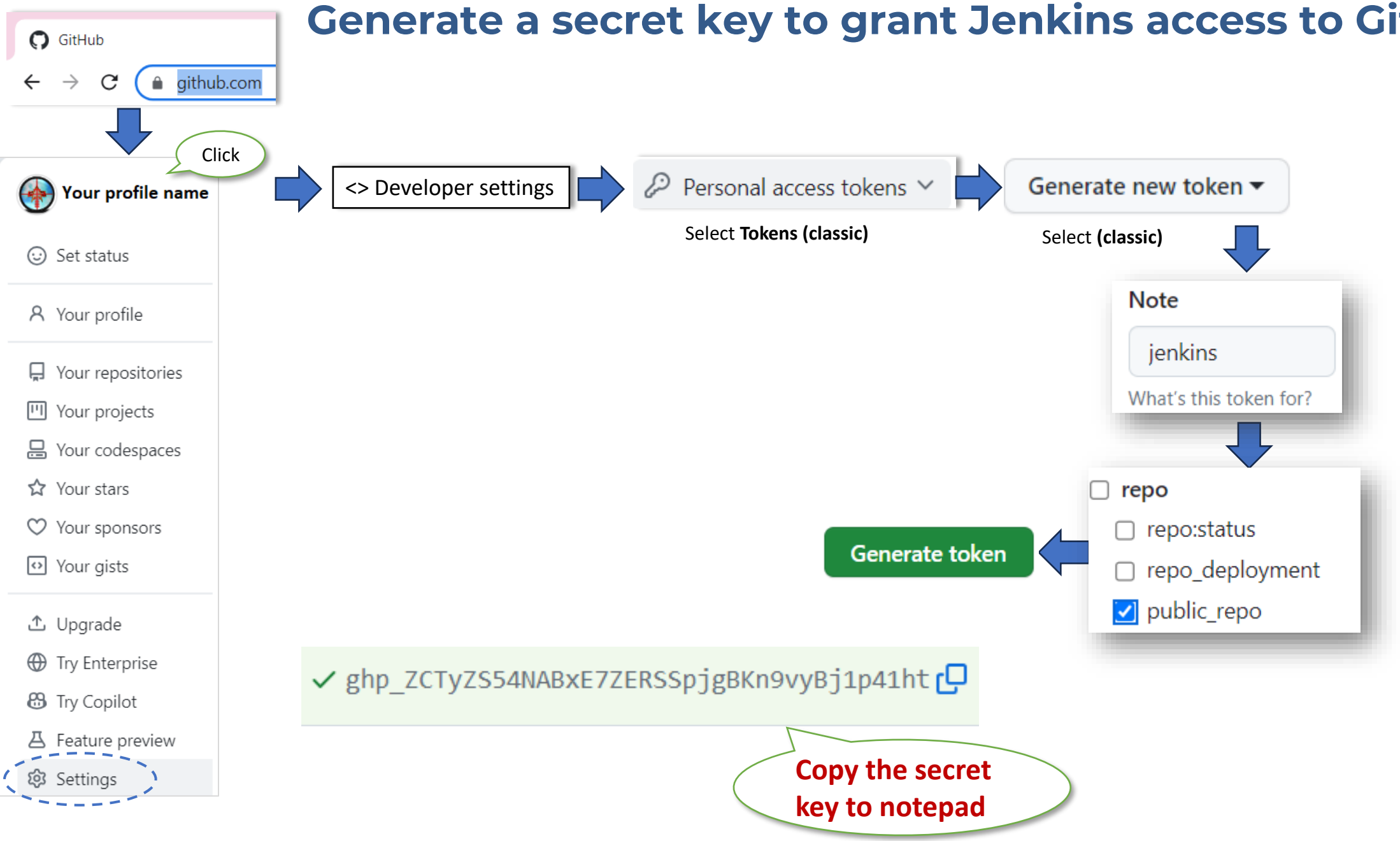


Scroll down until you find the `Maven` section





Generate a secret key to grant Jenkins access to GitHub



Create a repository for your code

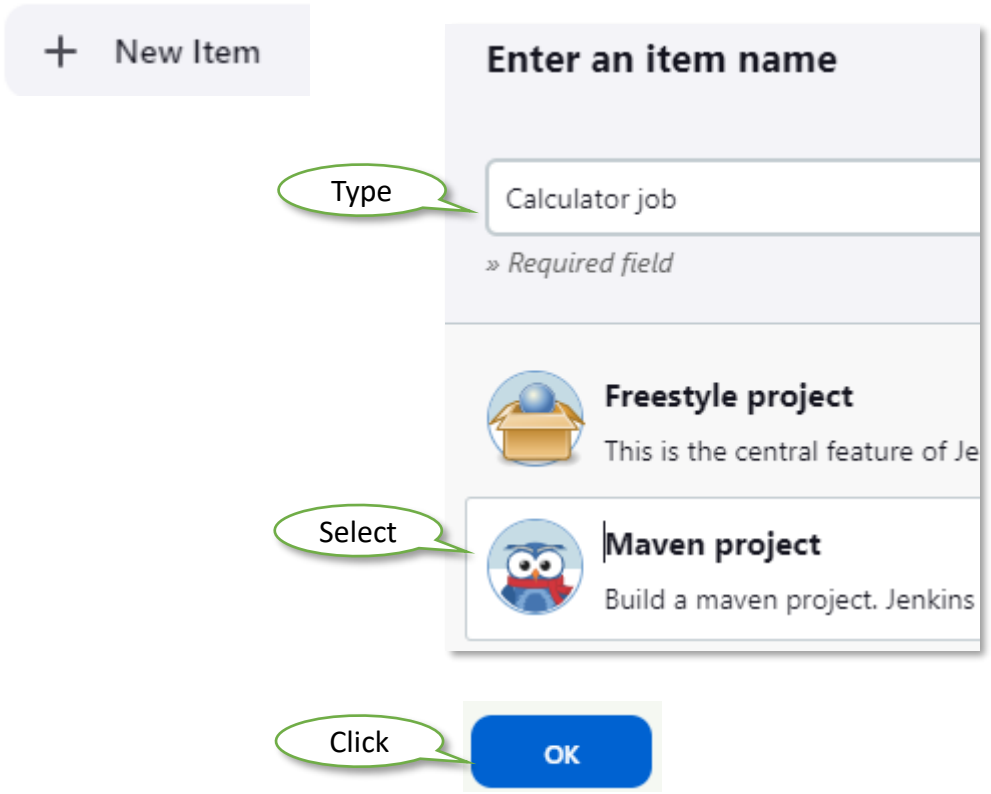
1. Copy the **calculator-jenkins** folder from the **Jenkins** folder to **C:\calculator-Jenkins**
2. Open a terminal window, and navigate to C:\calculator-Jenkins
   `cd c:\calculator-jenkins`
3. Initialise the folder as a new Git repo: `git init`
4. Add all of the files: `git add .`
5. Commit the files: `git commit -m "Initial commit"`
6. Create a new repository in GitHub and call it **calculator-Jenkins**
7. Type **git branch -M main** (in the terminal window)

   git remote add origin https://github.com/..../calculator-jenkins.git

   git push -u origin main

Copy the address

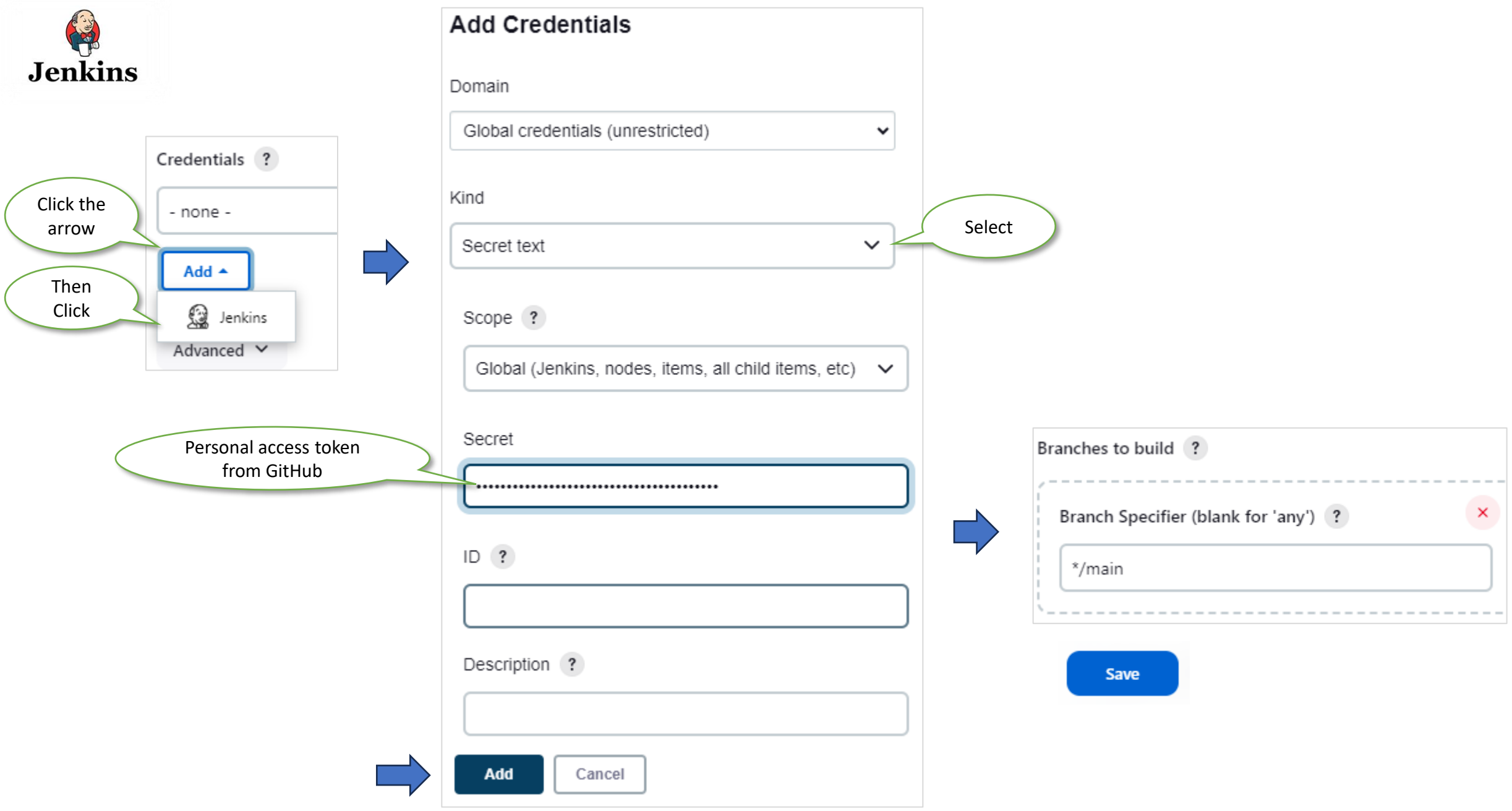
Create a new Jenkins job



On the screen below

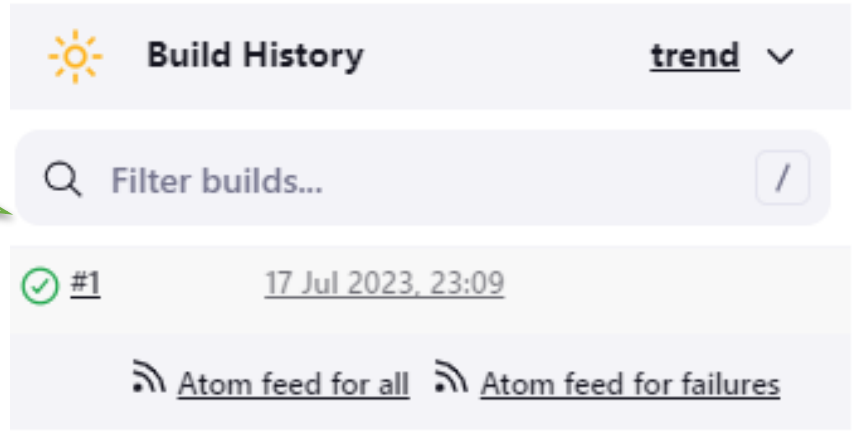


Continue on the next slide…


Jenkins
Credentials
- none -
Add
Jenkins
Advanced
Click the arrow
Then Click
Add Credentials
Domain
Global credentials (unrestricted)
Kind
Secret text
Select
Scope
Global (Jenkins, nodes, items, all child items, etc)
Secret
Personal access token from GitHub
ID
Description
Add
Cancel
Branches to build
Branch Specifier (blank for 'any')
*/main
Save

Jenkins



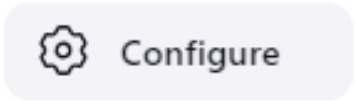


Click to see log

Console Output

```
Started by user admin
Running as SYSTEM
Building in workspace /var/jenkins_home/workspace/Calculator job
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/mikebaradaran/calculator-jenkins.git
 > git init /var/jenkins_home/workspace/Calculator job # timeout=10
Fetching upstream changes from https://github.com/mikebaradaran/calculator-jenkins.git
```

Configure Jenkins' Build Triggers

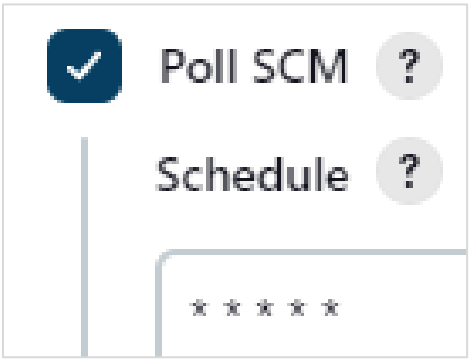
1. In the **Calculator job** window, select 

2. Click on 

3. Check the **`Poll SCM`** (Source Control Management) box

   Enter `* * * * *` into the **`Schedule`**

   to build every minute every time a commit is made to this repo



Let's test this on the next page

Run the Trigger (Terminal window)

1. In the C:\ drive **`/src/test/java/com/agile/calculator/CalculatorTest.java`**
2. Change the value of **`expected`** on line 17 to **`42`** (`makes the test fail`)
3. Save the file
4. Add and commit this file to the repo

   git add .

   git commit –m "change test result"
   git push

5. **Wait for a minute** (resist the urge to press the Build now button!)

This time the build fails, and the log indicates the reason for this failure

